gemini-conversation

Exported on: 6/29/2026, 9:00:04 PM

User

i want an explanation of the IAA algorithm enough so i know how to implement it in code

Gemini

The **Iterative Adaptive Approach (IAA)** is a powerful, non-parametric, user-parameter-free algorithm widely used in signal processing (such as radar, sonar, and spectral estimation) for high-resolution Direction-of-Arrival (DOA) or frequency estimation.

Unlike parametric methods (like MUSIC or ESPRIT), IAA does not require knowing the number of targets beforehand, and unlike the standard periodogram, it provides highly resolved peaks with suppressed sidelobes.

Here is the breakdown of the mathematics and the algorithmic steps designed to help you translate it directly into code.

The Mathematical Framework

Let $y \in \mathbb{C}^{M \times 1}$ be your vector of received signals (where M is the number of sensors or time samples).

We define a scanning grid of interest containing K potential look directions or frequencies. For this grid, we construct a steering matrix (or dictionary matrix) A:

$$A = [a_1, a_2, \dots, a_K] \in \mathbb{C}^{M \times K}$$

where a_k is the steering vector corresponding to the k-th grid point.

The objective of IAA is to estimate the complex amplitude x_k (and consequently the power $P_k = |x_k|^2$) at each grid point k. It achieves this by modeling the signal covariance matrix R as:

$$R = \sum_{m=1}^{K} P_m a_m a_m^H = A P A^H$$

where $P = \text{diag}(P_1, P_2, \dots, P_K)$ is a diagonal matrix containing the signal powers.

For each grid point k, IAA treats all other grid points as interference and noise. It minimizes a weighted least-squares criterion to find x_k:

$$x_k = \frac{a_k^H R^{-1} y}{a_k^H R^{-1} a_k}$$

Because R depends on the powers P_k, which depend on x_k, this must be solved **iteratively**.

Step-by-Step Implementation Algorithm

To implement this in code (e.g., using Python with NumPy or C++ with Eigen), follow these operational steps:

Step 1: Initialization (Iteration 0)

Before you can compute the adaptive covariance matrix R, you need an initial estimate of the power spectrum. This is typically done using a standard matched filter (periodogram):

For each grid point $k = 1, 2, \ldots, K$:

$$P_k^{(0)} = \left| \frac{a_k^H y}{a_k^H a_k} \right|^2$$

Step 2: The Iterative Loop

Repeat for a fixed number of iterations (typically 10 to 15 iterations are more than enough for convergence):

1. **Construct the Covariance Matrix R:** Compute the $M \times M$ matrix:

   $$R = APA^H$$

   Code Optimization Note: Since P is diagonal, this can be efficiently computed as `A @ np.diag(P) @ A.conj().T` or by scaling the columns of A by P before multiplying by A^H.

2. **Regularization (Crucial for Stability):** If $K > M$ or if some grid points have near-zero power, R can become ill-conditioned. Add a tiny identity regularization term to ensure invertibility:

   $$R = R + \epsilon I$$

   (where ϵ is a small positive scalar, e.g., $10^{-6} \cdot \text{trace}(R)/M$)

3. **Invert or Decompose R:** Compute R^{-1}. Code Optimization Note: Instead of explicitly calculating `inv(R)`, it is numerically more stable and efficient to compute the Cholesky decomposition of R and use back-substitution to solve the linear systems in the next step.

4. **Update Amplitudes and Powers:** For each grid point $k = 1, 2, \ldots, K$:
   - Compute the numerator: $N_k = a_k^H R^{-1} y$
   - Compute the denominator: $D_k = a_k^H R^{-1} a_k$
   - Update the power:

   $$P_k = \left| \frac{N_k}{D_k} \right|^2$$

Step 3: Termination

After the loop finishes, your vector $P = [P_1, P_2, \ldots, P_K]^T$ contains the finalized high-resolution power spectrum estimate.

Practical Implementation Tips

- **Vectorization:** If K is large, looping through each grid point k to compute P_k will slow down your code. You can vectorize the update step. Let $Z = R^{-1}A$. Then:
  - The denominators can be found via the column-wise dot product of A and Z: `D = np.sum(A.conj() * Z, axis=0)`
  - The numerators can be found via: `N = y.conj().T @ Z`
  - The power update becomes a simple element-wise operation: `P = np.abs(N / D) 2`
- **Storage Matrix Size:** Note that R is an $M \times M$ matrix. If your snapshot/sensor dimension M is very large, computing R^{-1} at every iteration becomes computationally expensive ($O(M^3)$). IAA is most computationally efficient when M is modest.

User

provide a matlab and c++ implementation

Gemini

Here are the vectorized MATLAB and C++ (using the Eigen library) implementations of the IAA algorithm based on the math outlined previously.

Both implementations use **Cholesky decomposition** (LL^H) instead of explicit matrix inversion for numerical stability and speed.

1. MATLAB Implementation

This script leverages MATLAB's column-wise operations to eliminate the loop over the K grid points, making it highly efficient.

Matlab

```matlab
function P = iaa_algorithm(y, A, num_iters)
    % IAA_ALGORITHM Iterative Adaptive Approach for Spectral/DOA Estimation
    % Inputs:
    %   y         : Received signal vector (M x 1)
    %   A         : Steering/dictionary matrix (M x K)
    %   num_iters : Number of iterations (typically 10-15)
    % Output:
    %   P         : Estimated power spectrum vector (K x 1)

    [M, K] = size(A);

    % --- Step 1: Initialization (Periodogram) ---
    % Compute denominator for each grid point: ||a_k||^2
    den_init = sum(abs(A).^2, 1).';
    % Matched filter outputs: a_k^H * y
    num_init = A' * y;
    % Initial Power
    P = (abs(num_init) ./ den_init).^2;
```

```
    % Small regularization factor identity matrix
    I = eye(M);

    % --- Step 2: Iterative Loop ---
    for iter = 1:num_iters
        % Construct the covariance matrix R = A * diag(P) * A^H
        % Optimized by scaling columns of A directly
        R = A * (P .* A');

        % Regularization to prevent ill-conditioning
        epsilon = 1e-6 * trace(R) / M;
        R = R + epsilon * I;

        % Solve R \ A and R \ y using Cholesky decomposition for stability
        % R = L * L' -> MATLAB's backslash uses efficient solvers automatically
        Z = R \ A;       % M x K matrix (R^-1 * A)
        z_y = R \ y;     % M x 1 vector (R^-1 * y)

        % Vectorized update of numerators and denominators
        num = A' * z_y;                % K x 1: a_k^H * R^-1 * y
        den = sum(conj(A) .* Z, 1).';  % K x 1: a_k^H * R^-1 * a_k

        % Update Power Spectrum
        P = (abs(num) ./ abs(den)).^2;
    end
end
```

2. C++ Implementation (with Eigen)

This implementation uses the **Eigen** library, employing a row-major configuration for the steering matrix where appropriate, or column-wise vectorization to maximize cache locality.

C++

```cpp
#include <iostream>
#include <Eigen/Dense>

// Type definitions for complex matrices and vectors
typedef Eigen::Matrix<std::complex<double>, Eigen::Dynamic, Eigen::Dynamic> MatrixXcd;
typedef Eigen::Matrix<std::complex<double>, Eigen::Dynamic, 1> VectorXcd;
typedef Eigen::VectorXd VectorXd;

/**
 * @brief Computes the Iterative Adaptive Approach (IAA) spectrum.
 * * @param y         Received signal vector (M x 1)
 * @param A         Steering matrix (M x K)
 * @param num_iters Number of iterations
 * @return VectorXd Estimated power spectrum (K x 1)
 */
VectorXd iaa_algorithm(const VectorXcd& y, const MatrixXcd& A, int num_iters) {
    int M = A.rows();
    int K = A.cols();

    // --- Step 1: Initialization (Periodogram) ---
    VectorXd P(K);
    VectorXd den_init = A.colwise().squaredNorm(); // ||a_k||^2 for each column
    VectorXcd num_init = A.adjoint() * y;          // A^H * y

    for (int k = 0; k < K; ++k) {
        double abs_num = std::abs(num_init(k));
        P(k) = std::pow(abs_num / den_init(k), 2);
    }

    MatrixXcd I = MatrixXcd::Identity(M, M);
```

```cpp
    // --- Step 2: Iterative Loop ---
    for (int iter = 0; iter < num_iters; ++iter) {
        // Compute R = A * P * A^H
        // A. some columns scaled by P: A * P.asDiagonal()
        MatrixXcd R = A * P.asDiagonal() * A.adjoint();

        // Regularization: R = R + epsilon * I
        double epsilon = 1e-6 * R.trace().real() / M;
        R += epsilon * I;

        // Use Cholesky Decomposition (LLT / LDLT) for solving linear systems
        // Since R is Hermitian positive definite, LLT is highly efficient
        Eigen::LLT<MatrixXcd> llt(R);

        // Z = R^-1 * A  => Solve R * Z = A
        MatrixXcd Z = llt.solve(A);
        // z_y = R^-1 * y => Solve R * z_y = y
        VectorXcd z_y = llt.solve(y);

        // Numerator vector: A^H * R^-1 * y
        VectorXcd num = A.adjoint() * z_y;

        // Denominator vector: diag(A^H * R^-1 * A)
        // Can be evaluated efficiently via element-wise multiplication and column sums
        for (int k = 0; k < K; ++k) {
            std::complex<double> den_k = A.col(k).dot(Z.col(k)); // internal conjugate-dot

            // Update power element-wise
            P(k) = std::pow(std::abs(num(k)) / std::abs(den_k), 2);
        }
    }

    return P;
}
```

Compilation Notes for C++

To compile the C++ code, ensure your compiler optimization flags are turned on (e.g., `-O3` with GCC or Clang) so Eigen can vectorize loops under the hood:

Bash

```
g++ -O3 -I /path/to/eigen main.cpp -o iaa_estimation
```

User

write a test for the matlab version

Gemini

Here is a complete, self-contained MATLAB test script. It generates a synthetic signal consisting of two closely spaced targets (representing two distinct frequencies or Angles of Arrival) embedded in White Gaussian Noise, runs the `iaa_algorithm` function, and plots the resulting high-resolution spectrum.

Test Script: `test_iaa.m`

Save the following code in the same directory as your `iaa_algorithm.m` file and run it.

Matlab

```
% test_iaa.m
% Test script to validate the Iterative Adaptive Approach (IAA) implementation

clear; clc; close all;

%% 1. Simulation Parameters
M = 32;          % Number of sensors / samples
SNR_dB = 20;     % Signal-to-Noise Ratio in dB

% True target normalized frequencies (normalized to [-0.5, 0.5])
% We put them close together to test IAA's high-resolution capability
true_freqs = [-0.05, 0.02];
amplitudes = [1.0, 0.8] .* exp(1i * [pi/4, -pi/3]); % Complex amplitudes

%% 2. Generate Signal and Scanning Grid
n = (0:M-1).';   % Time/spatial index vector (M x 1)

% Generate clean signal
y_clean = zeros(M, 1);
for idx = 1:length(true_freqs)
```

```
    y_clean = y_clean + amplitudes(idx) * exp(1i * 2 * pi * true_freqs(idx) * n);
end

% Add Complex White Gaussian Noise
noise_power = 10^(-SNR_dB / 10) * (norm(y_clean)^2 / M);
noise = sqrt(noise_power / 2) * (randn(M, 1) + 1i * randn(M, 1));
y = y_clean + noise;

% Define scanning grid (K potential frequencies)
K = 500;
grid_freqs = linspace(-0.5, 0.5, K);

% Construct Steering Matrix A (M x K)
A = exp(1i * 2 * pi * n * grid_freqs);

%% 3. Run IAA vs Standard Periodogram (Iteration 0)
num_iters = 15;
P_iaa = iaa_algorithm(y, A, num_iters);

% Compute standard periodogram for baseline comparison
P_per = (abs(A' * y) ./ sum(abs(A).^2, 1).').^2;

%% 4. Plot Results
figure('Position', [100, 100, 900, 500]);
hold on;

% Plot Periodogram (Normalized to dB)
plot(grid_freqs, 10*log10(P_per / max(P_per)), 'r--', 'LineWidth', 1.5);

% Plot IAA Spectrum (Normalized to dB)
plot(grid_freqs, 10*log10(P_iaa / max(P_iaa)), 'b-', 'LineWidth', 2);

% Mark Ground Truth positions
for idx = 1:length(true_freqs)
    xline(true_freqs(idx), 'k:', 'LineWidth', 1.5, 'HandleVisibility', 'off');
```

```
    stem(true_freqs(idx), 0, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 6);
end

% Formatting
grid on;
xlim([-0.5, 0.5]);
ylim([-40, 5]);
xlabel('Normalized Frequency');
ylabel('Normalized Power Spectrum (dB)');
title(sprintf('IAA vs Periodogram Performance (M = %d, SNR = %d dB)', M, SNR_dB));
legend('Standard Periodogram', ['IAA (', num2size(num_iters), ' Iters)'], 'True Targets', 'Location', 'Best');

% Helper string conversion for title compatibility
function s = num2size(n)
    s = num2str(n);
end
```

What to Expect when Running the Test

When you run this script, it will pop open a figure window comparing the two methods:

- **The Standard Periodogram (Red Dashed Line):** Will show broad mainlobes and high sidelobes. If the targets were any closer, the periodogram would merge them into a single blob.
- **The IAA Spectrum (Blue Solid Line):** Will show incredibly sharp, distinct peaks precisely at -0.05 and 0.02, with the noise floor heavily suppressed down to -30 dB or lower.